



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Outline

Part 0. Overview

Part 1. Software Processes

Part 2. Modeling

❖ **Chap 7. Understanding Requirements**

❖ **Chap 8-10. Requirements Modeling**

❖ **Chap 11. Design Concepts**

❖ **Chap 12-14. Design Methods**

Part 3. Quality Assurance

Part 4. Software Project Management

Chap 12. Behavioral Modeling

- ❖ **1. What is Architecture**
- ❖ **2. Architectural Styles**
- ❖ **3. Architectural Patterns**
- ❖ **4. Architectural Design**
- ❖ **5. Architecture Reviews**
- ❖ **6. Agility and Architecture**

What is Architecture?

❖ The software architecture of a program or computing system is **the structure or structures of the system**, which comprise the **software components, the externally visible properties** of those components, and **the relationships** among them.

— *Bass. et al.*

1. 什么是架构

程序或计算系统的软件架构是系统的一个或多个结构，它包括软件组件、这些组件的外部可见属性以及它们之间的关系

Why Architecture?

The architecture is not the operational software. Rather, it is a representation that enables a software engineer to:

- (1) **analyze the effectiveness** of the design in meeting its stated requirements,
- (2) **consider architectural alternatives** at a stage when making design changes is still relatively easy, and
- (3) **reduce the risks** associated with the construction of the software.

2. 架构有什么用

- 分析设计在满足其规定要求方面的有效性
- 在进行设计更改仍然相对而言的阶段考虑架构替代方案容易
- 降低与软件构建相关的风险

Why is Architecture Important?

- ❖ Representations of software architecture are an enabler **for communication between all parties (stakeholders)** interested in the development of a computer-based system.
- ❖ **The architecture highlights early design decisions** that will have a profound impact on all software engineering work that follows and, as important, on the ultimate success of the system as an operational entity.
- ❖ Architecture “**constitutes a relatively small, intellectually graspable mode** of how the system is structured and how its components work together” [BAS03].

3. 架构的重要性

- 软件体系结构的表示是对基于计算机的系统的开发感兴趣的所有各方（利益相关者）之间进行交流的促成因素
- 架构突出了早期的设计决策，这些决策将对随后的所有软件工程项目产生深远的影响，对系统作为一个操作实体的最终成功产生深远影响
- 架构“构成了一种相对较小的、智力上可掌握的系统结构，便于理解

4. 架构描述——记录架构的产品集合 (IEEE)

Architectural Descriptions

- ❖ The IEEE Computer Society has proposed IEEE-Std-1471-2000, *Recommended Practice for Architectural Description of Software-Intensive System*, [IEE00]
 - to **establish a conceptual framework and vocabulary** for use during the design of software architecture,
 - to **provide detailed guidelines** for representing an architectural description, and
 - to encourage sound architectural design practices.
- ❖ The IEEE Standard defines an *architectural description* (AD) as a “**a collection of products to document an architecture.**”
 - The description itself is represented using multiple views, where each *view* is “a representation of a whole system from the perspective of a related set of [stakeholder] concerns.”

Architectural Genres（类型）

- ❖ **Genre** implies a specific category within the overall software domain.
- ❖ Within each category, you encounter a number of subcategories.
 - For example, within the genre of *buildings*, you would encounter the following general **styles**: houses, condos（单元楼）, apartment buildings, office buildings, industrial building, warehouses, and so on.
 - Within each general style, more specific styles might apply. Each style would have a structure that can be described using a set of predictable patterns.

- **类型**意味着在整体软件领域内的一个特定类别。

- 在每个类别中，你会遇到许多子类别。

- 例如，在建筑类别中，你会遇到以下一般风格：房屋、公寓楼（单元楼）、公寓建筑、办公楼、工业建筑、仓库等。

- 在每种一般风格中，可能还会有更具体的风格。每种风格都有一套可预测的模式来描述其结构。

Architectural Styles

- ❖ The same problems can be solved in different ways, but we usually adopt a specific mode according to our experience.
 - ❖ **Software style** is a method to form a structure and structure relationship with integrity, universality and generality.
 - ❖ Therefore, **software style is a specific basic structure.**
- 相同的问题可以通过不同的方法解决，但我们通常根据经验采用一种特定的模式。
 - 软件风格是一种形成具有完整性、通用性和普遍性的结构和结构关系的方法。
 - 因此，软件风格是一种特定的基本结构。

Architectural Styles

❖ Each style describes a system category that encompasses:

- (1) a **set of components** (e.g., a database, computational modules) that perform a function required by a system,
- (2) a **set of connectors** that enable “communication, coordination and cooperation” among components,
- (3) **constraints** that define how components can be integrated to form the system, and
- (4) **semantic models** that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

○ 架构风格定义了

- 一组执行系统所需功能的构件
- 一组连接器，可以实现“通信、协调和组件之间的合作”
- 定义组件如何集成以形成系统的约束
- 语义模型，使设计人员能够通过分析系统组成部分的已知特性来理解系统的整体属性

Architectural Styles

- 架构风格的种类

- 数据流架构——批处理、管道和过滤器
- 调用和返回架构——主程序/子程序、面向对象系统、分层系统
- 独立组件架构——事件系统、触发器、监视器
- 虚拟机架构——解释器，基于规则的系统
- 仓库架构——数据库系统，黑板系统等

- ❖ **Data flow architectures**

- batch processing, pipes and filters

- ❖ **Call and return architectures**

- main program / subroutines, object-oriented system, layered system;

- ❖ **Independent component architectures**

- event system

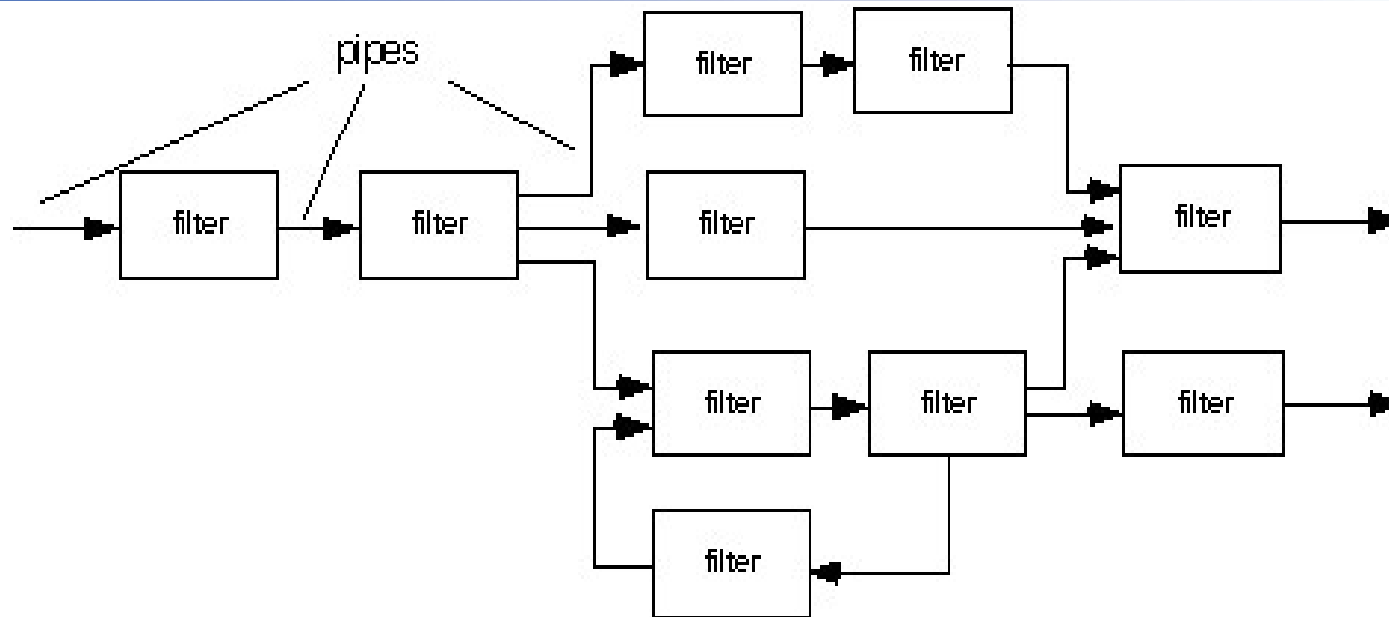
- ❖ **Virtual machine architectures**

- interpreter, rule-based system

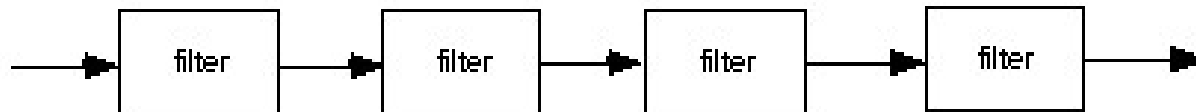
- ❖ **Repository architectures**

- database system, blackboard system, etc.

Data Flow Architecture

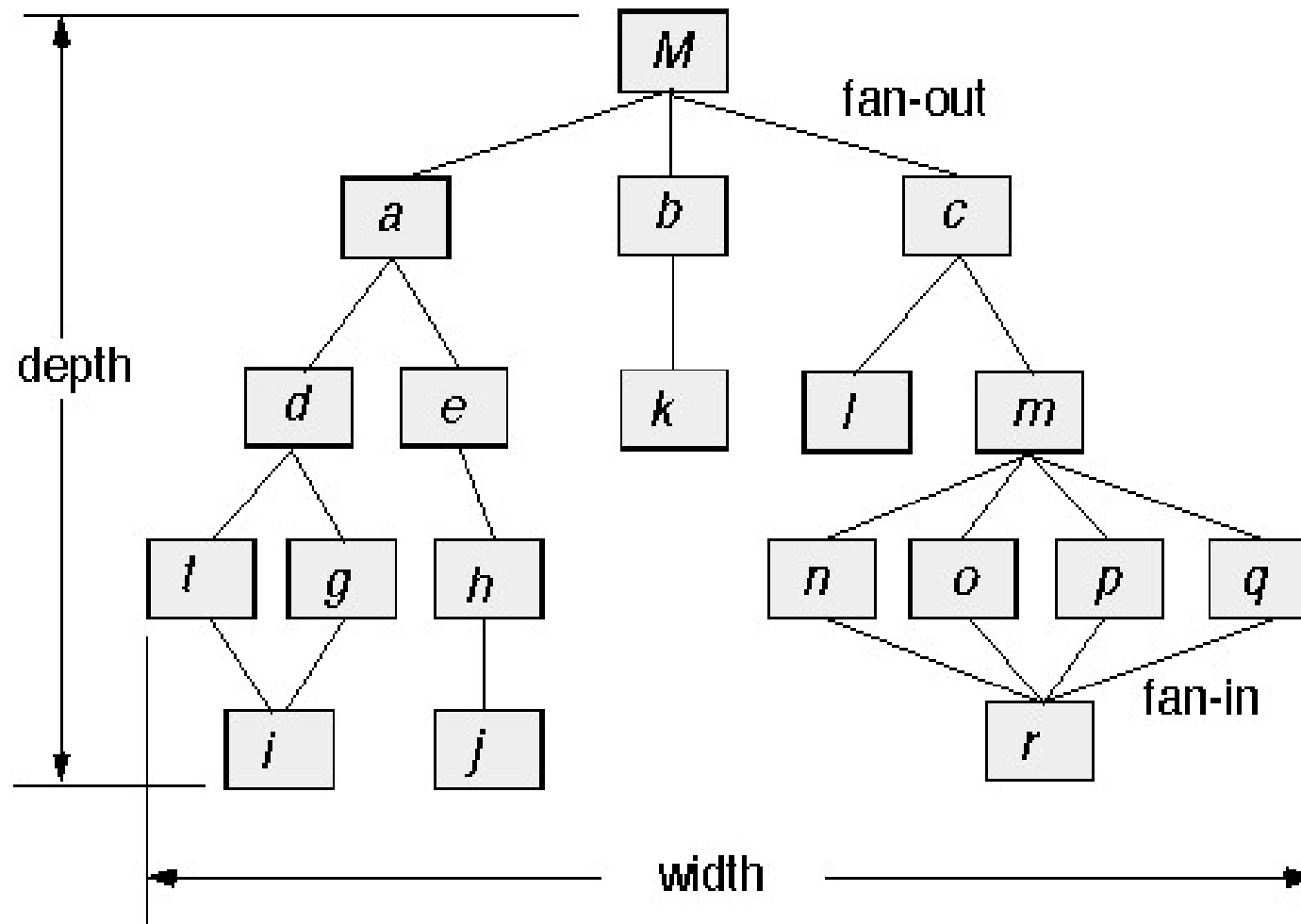


(a) pipes and filters

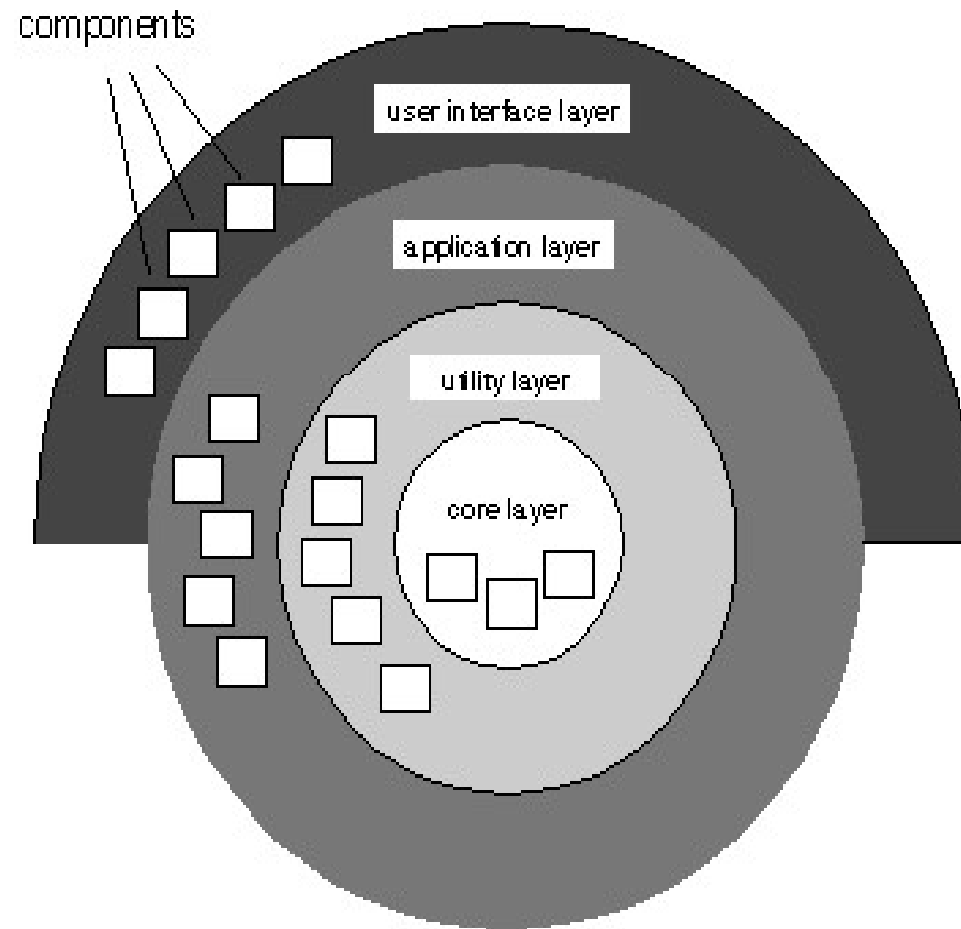


(b) batch sequential

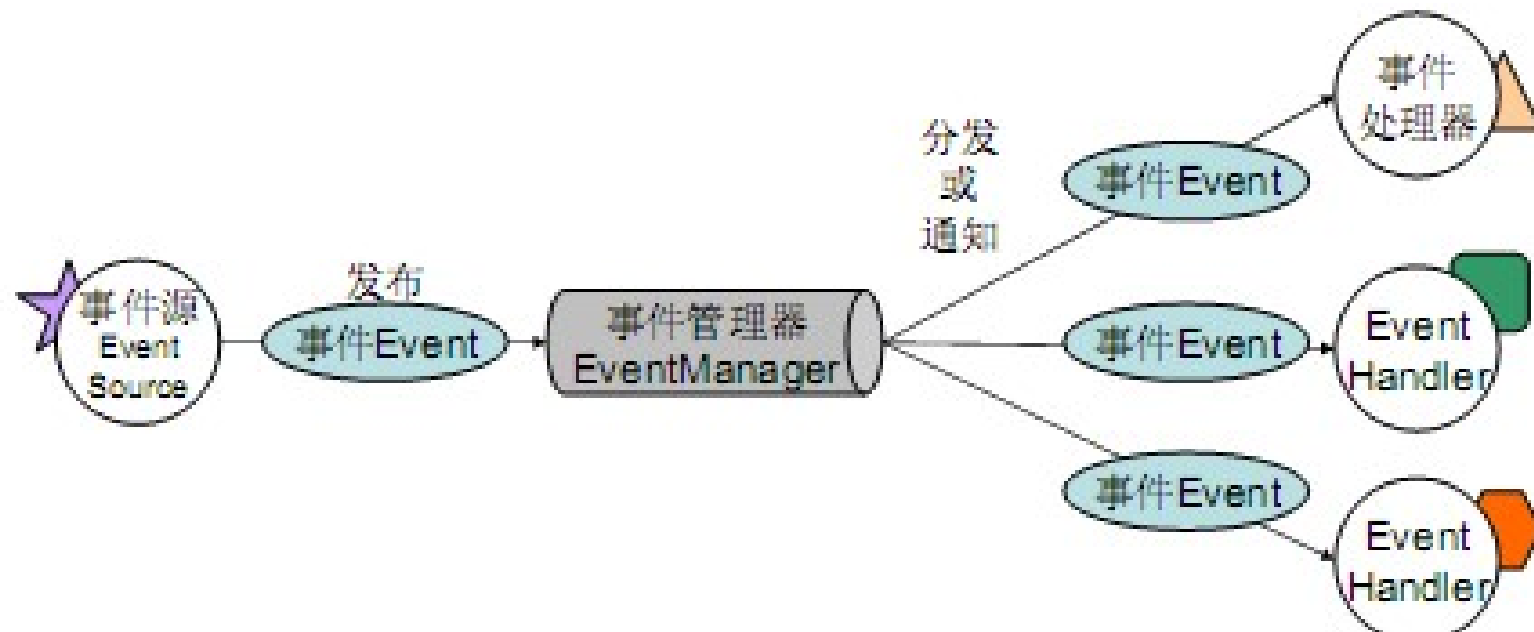
Call and Return Architecture- main program / subroutines



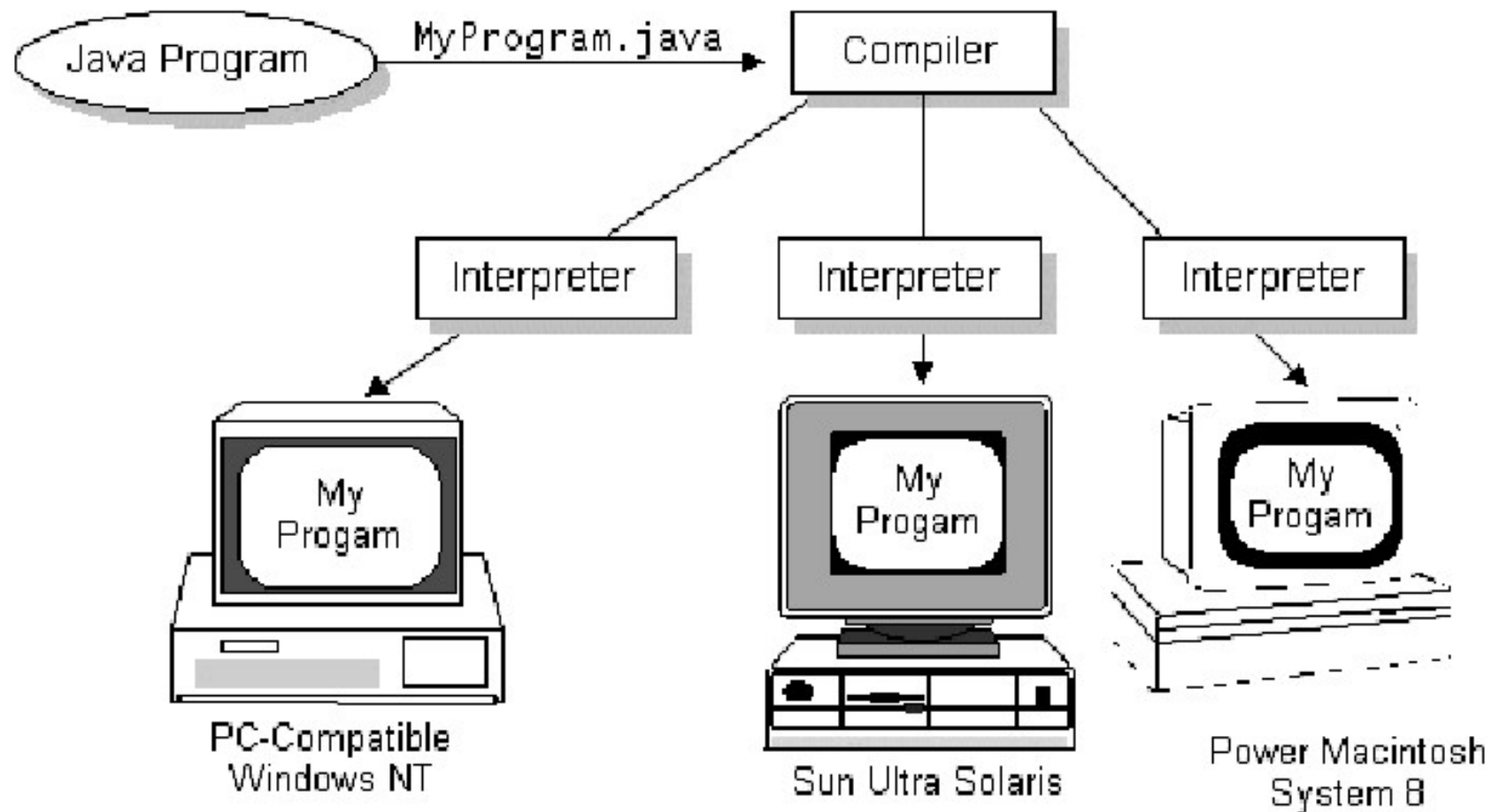
Call and Return Architecture- Layered Architecture



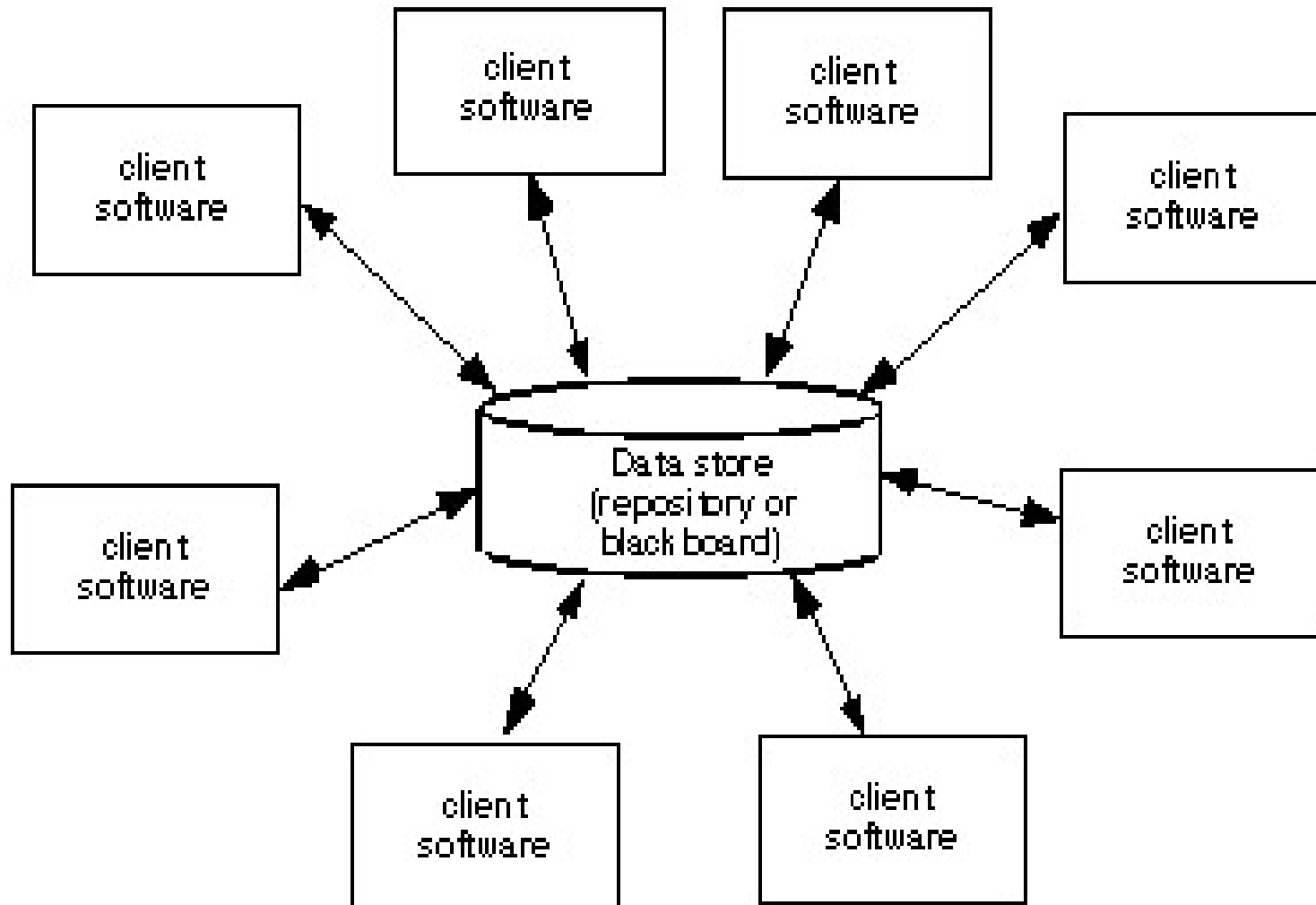
Independent component architectures- event system



Virtual machine architectures- interpreter



Repository Architecture



架构模式

Architectural Patterns

- ❖ **Concurrency**—applications must handle multiple tasks in a manner that simulates parallelism .
 - *operating system process management* pattern
 - *task scheduler* pattern
- ❖ **Persistence**—Data persists if it survives past the execution of the process that created it. Two patterns are common:
 - a *database management system* pattern that applies the storage and retrieval capability of a DBMS to the application architecture
 - an *application level persistence* pattern that builds persistence features into the application architecture
- ❖ **Distribution**— the manner in which systems or components within systems communicate with one another in a distributed environment.
 - A *broker* acts as a ‘middle-man’ between the client component and a server component.

- 并发——并行执行任务（OS进程管理、任务调度）
- 持久性——数据在创立之后持久存在（数据库）
- 分发——系统在分布式环境中通信（代理）

架构设计

Architectural Design

- ❖ The software must be placed into context
 - the design should define the external entities (other systems, devices, people) that the software interacts with and the nature of the interaction
- ❖ A set of architectural archetypes should be identified
 - An *archetype* is an abstraction (similar to a class) that represents one element of system behavior
- ❖ The designer specifies the structure of the system by defining and refining software components that implement each archetype

- 软件必须置于上下文中

- 应确定一组架构原型

- 设计应定义软件交互的外部实体（其他系统、设备、人员）及其交互的性质

- 原型是一个抽象（类似于类），代表系统行为的一个元素

- 设计人员通过定义和完善软件组件来指定系统的结构，以实现每个原型

Architectural Design Steps

1. Design preliminary software architecture
2. Reuse available software resources
3. Refined architecture design
 - (1) Select the **public infrastructure** for the software architecture;
 - Database, Message oriented middleware, etc.
 - For some basic services, the infrastructure has already provided relevant functions
 - For those basic services whose infrastructure does not provide relevant functions and needs secondary development, the software architect needs to make further development

Architectural Design Steps

1. Design preliminary software architecture
2. Reuse available software resources
3. Refined architecture design
4. Design deployment model
5. Architecture documentation

架构设计步骤

1. 设计初步的软件架构
2. 重用可用的软件资源
3. 细化架构设计
4. 设计部署模型
5. 架构文档编制

Architectural Design Steps

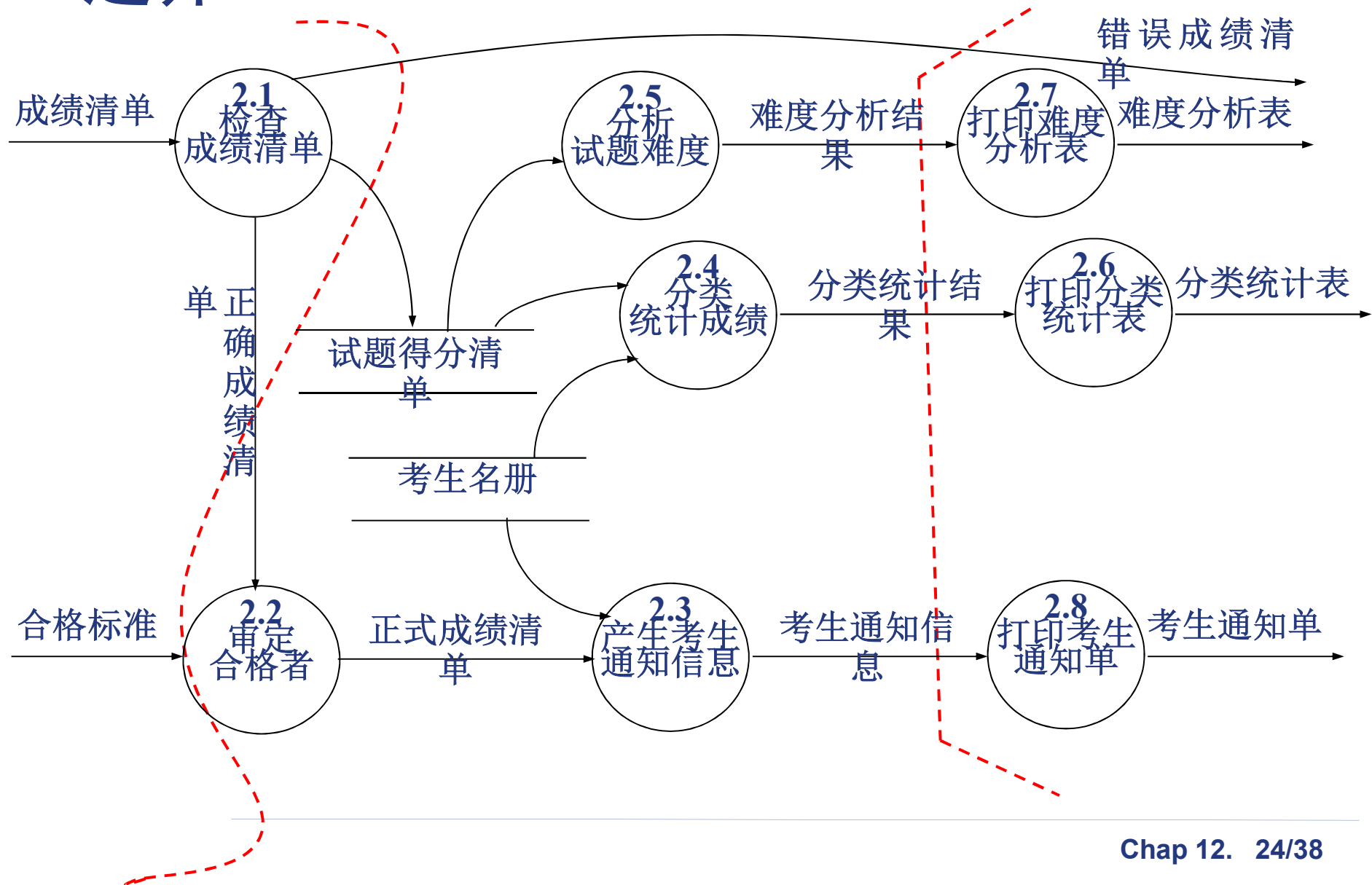
1. Design preliminary software architecture

- **Target:** Based on the requirements of the software system, give the overall solution of the software.
- **Steps:**
 - (1) **Identify key requirements:** Functional, Quality, Constrains, Risks
 - (2) **Build a preliminary software architecture:**
 - Refer to different architecture styles
 - Combine your own experience

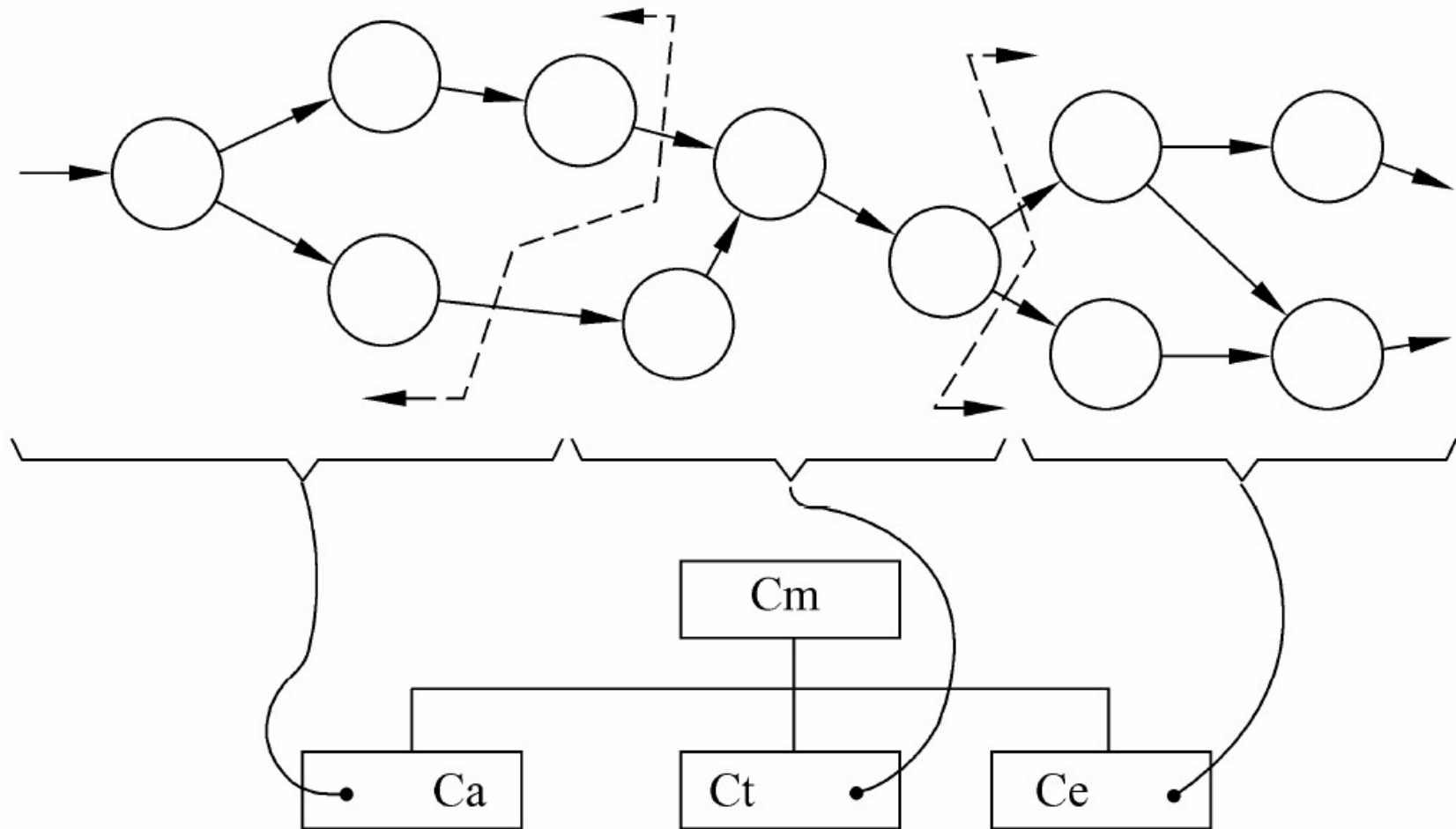
1. 设计初步软件架构

- 目标：根据软件系统的要求，提供整体解决方案。
- 步骤：
 - 1. 确定关键需求：功能性、质量、约束、风险
 - 2. 构建初步软件架构：
 - 参考不同的架构样式
 - 结合自己的经验

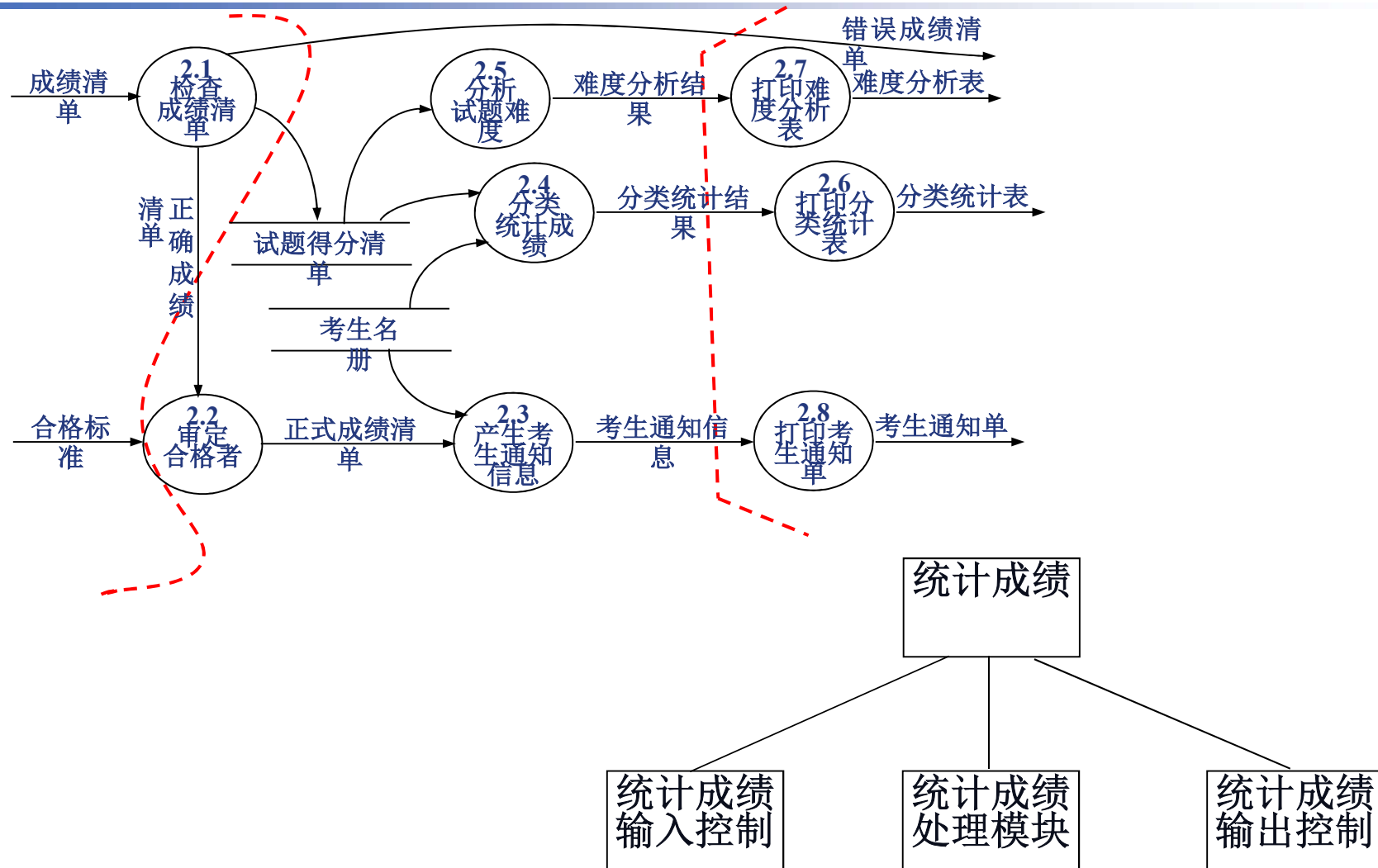
示例：统计成绩子图的输入、输出流边界



Structural Design

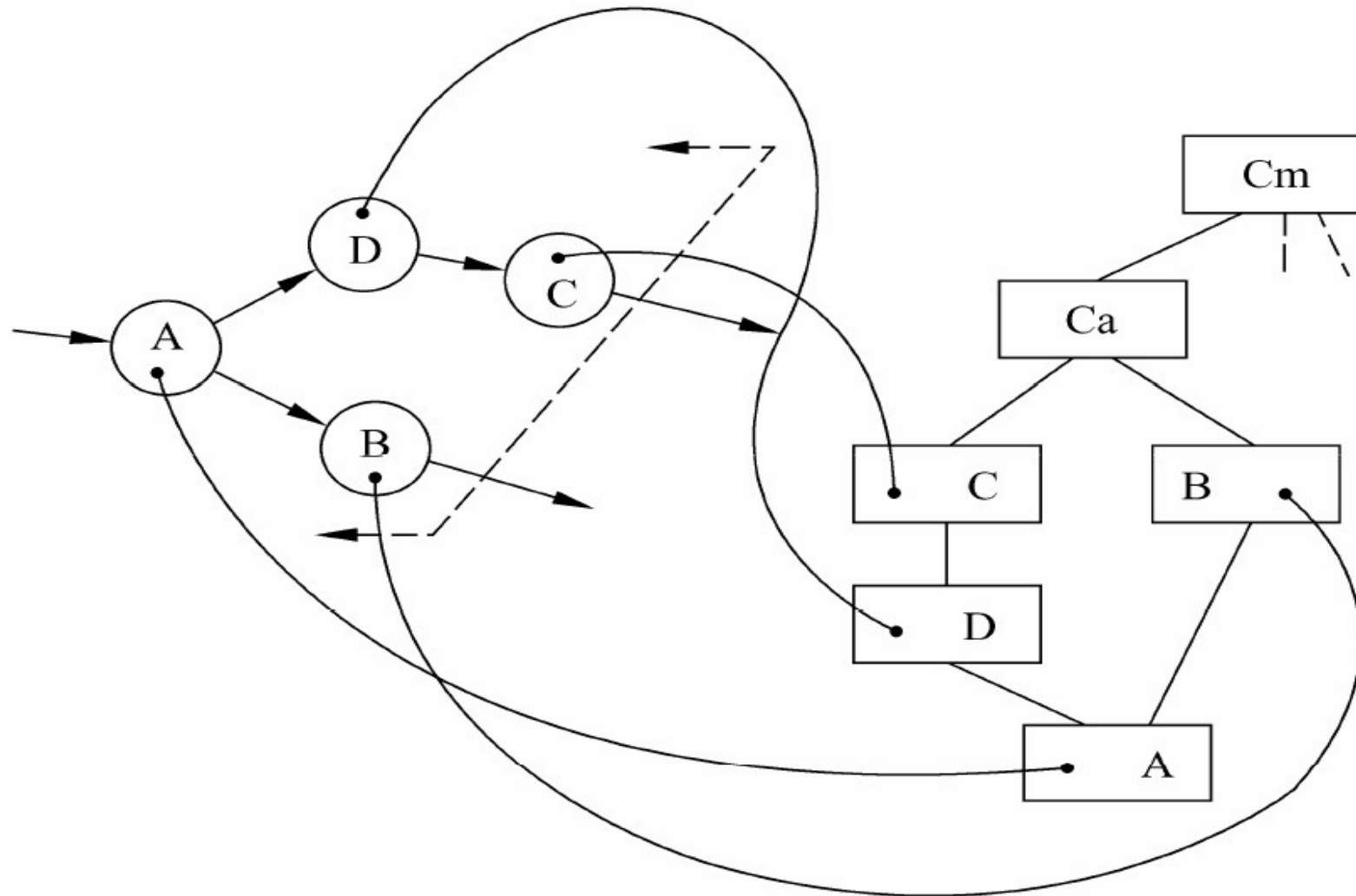


示例：统计成绩子图的第一级分解

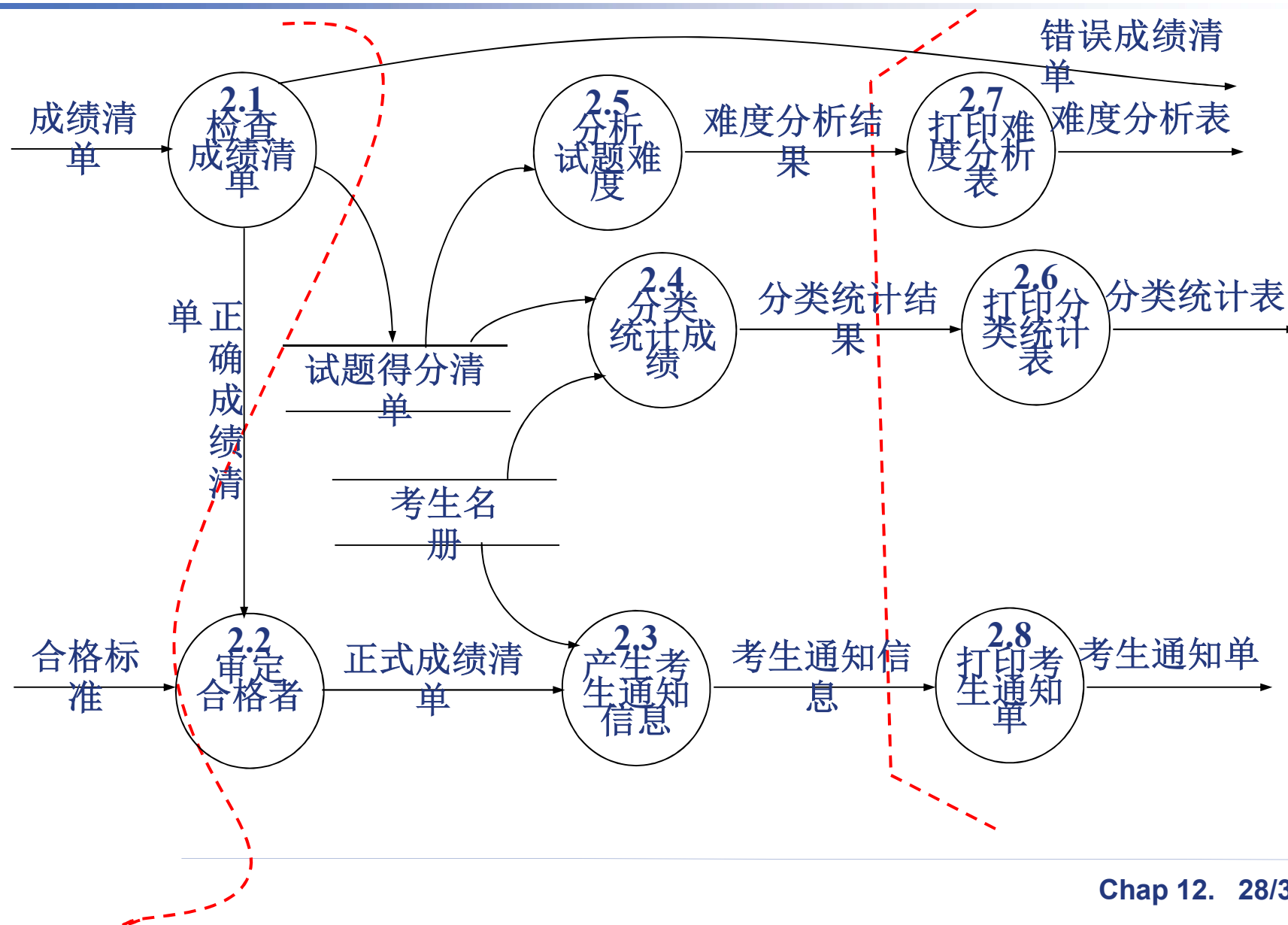


“统计成绩”第一级分解的结构图

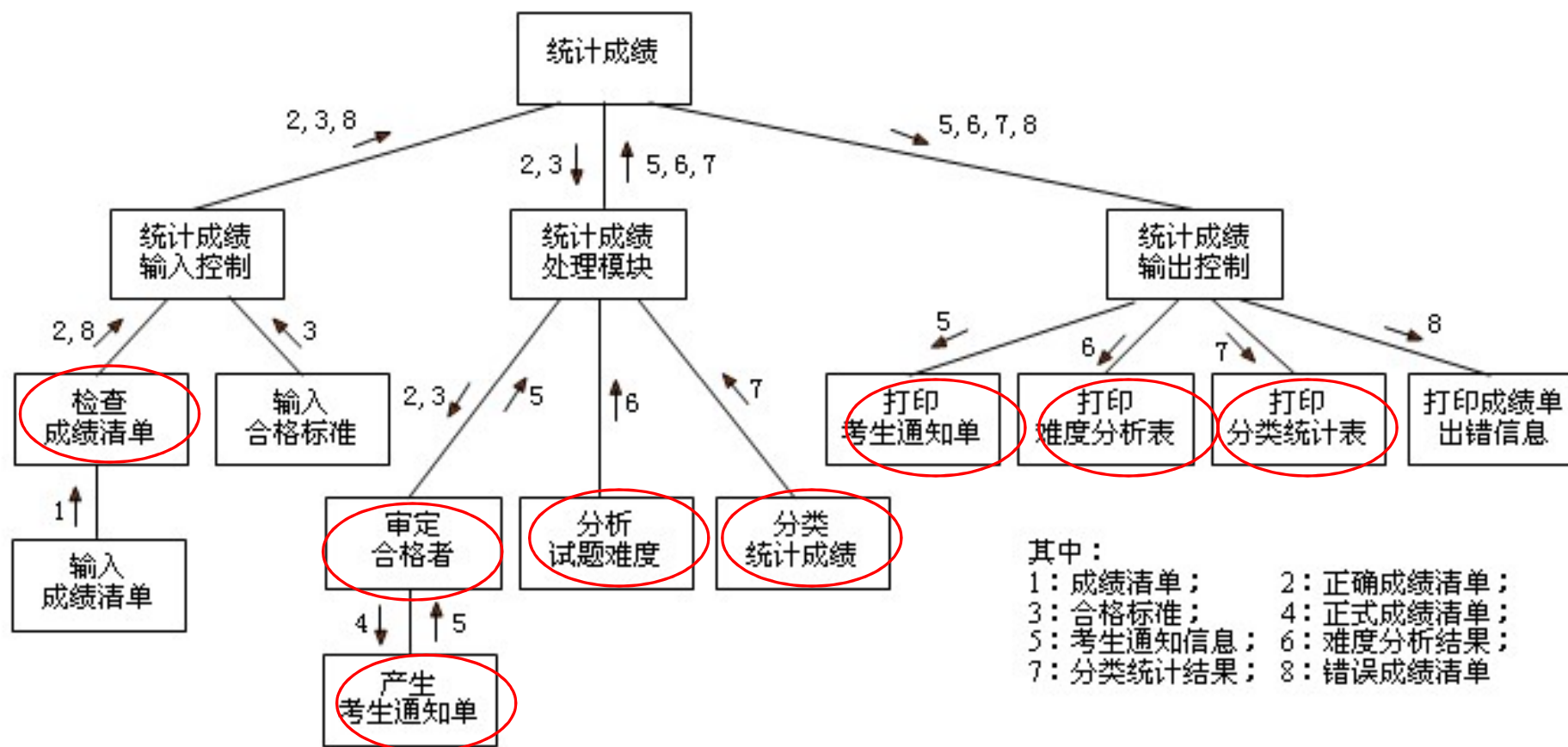
Structural Design



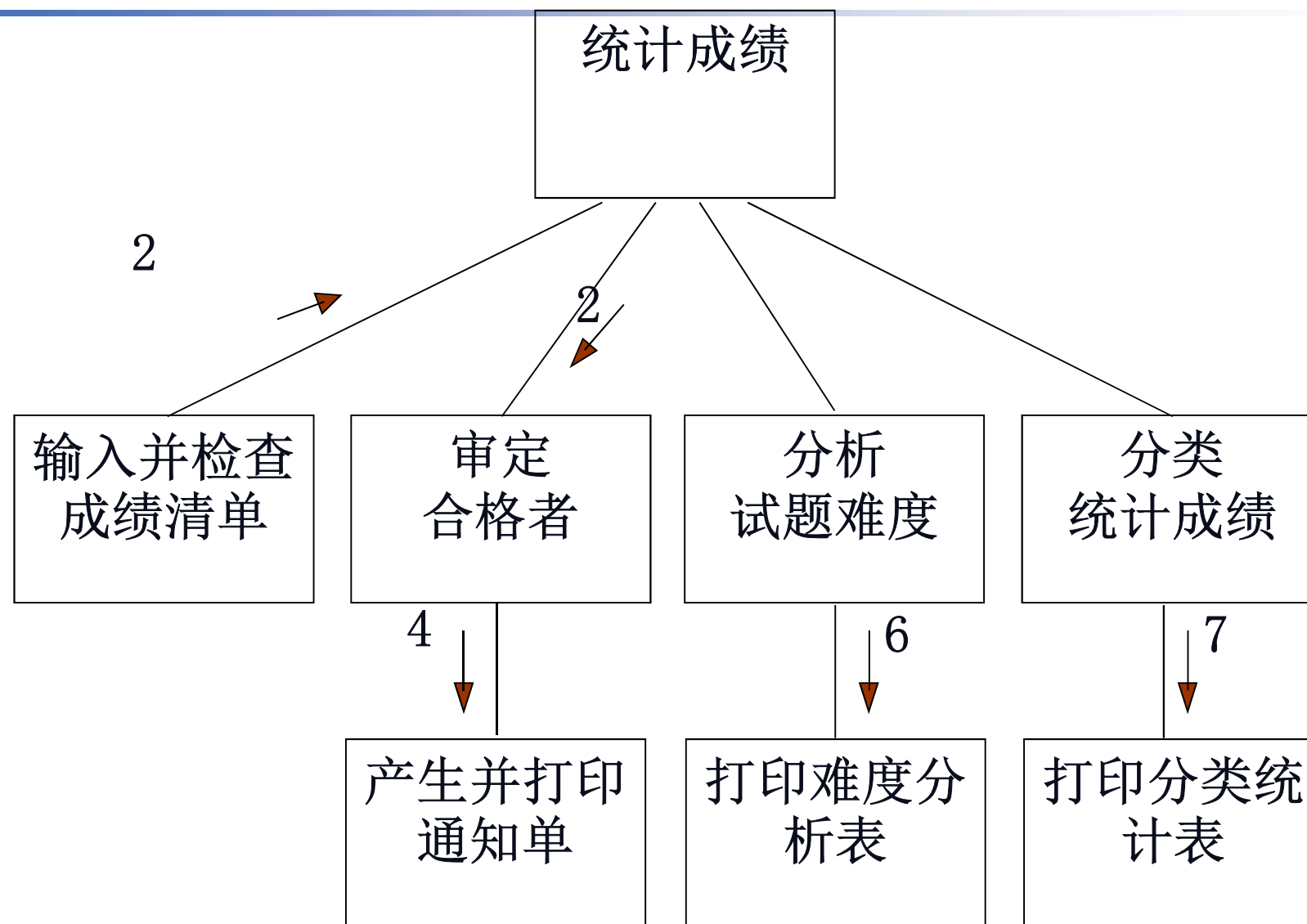
示例2：“统计成绩”数据流图



“统计成绩”第二级分解的结构图



“统计成绩” 结构图改进



Architectural Design Steps

1. Design preliminary software architecture
2. Reuse available software resources
 - Find **coarse-grained**（粗粒度） **software resources** to realize the various components and functions in the software system.
 - Reusable software development kit (SDK)
 - Cloud services on the Internet
 - Open source software
 - Legacy systems
 - **Select** appropriate software products from them, clearly **define their interaction mode and interface** with the target software, and introduce them into the target software.

2. 重用可用的软件资源

- 找到粗粒度的软件资源，实现软件系统中的各种组件和功能。
- 选择合适的软件产品，清楚地定义它们与目标软件的交互模式和接口，并将它们引入目标软件。

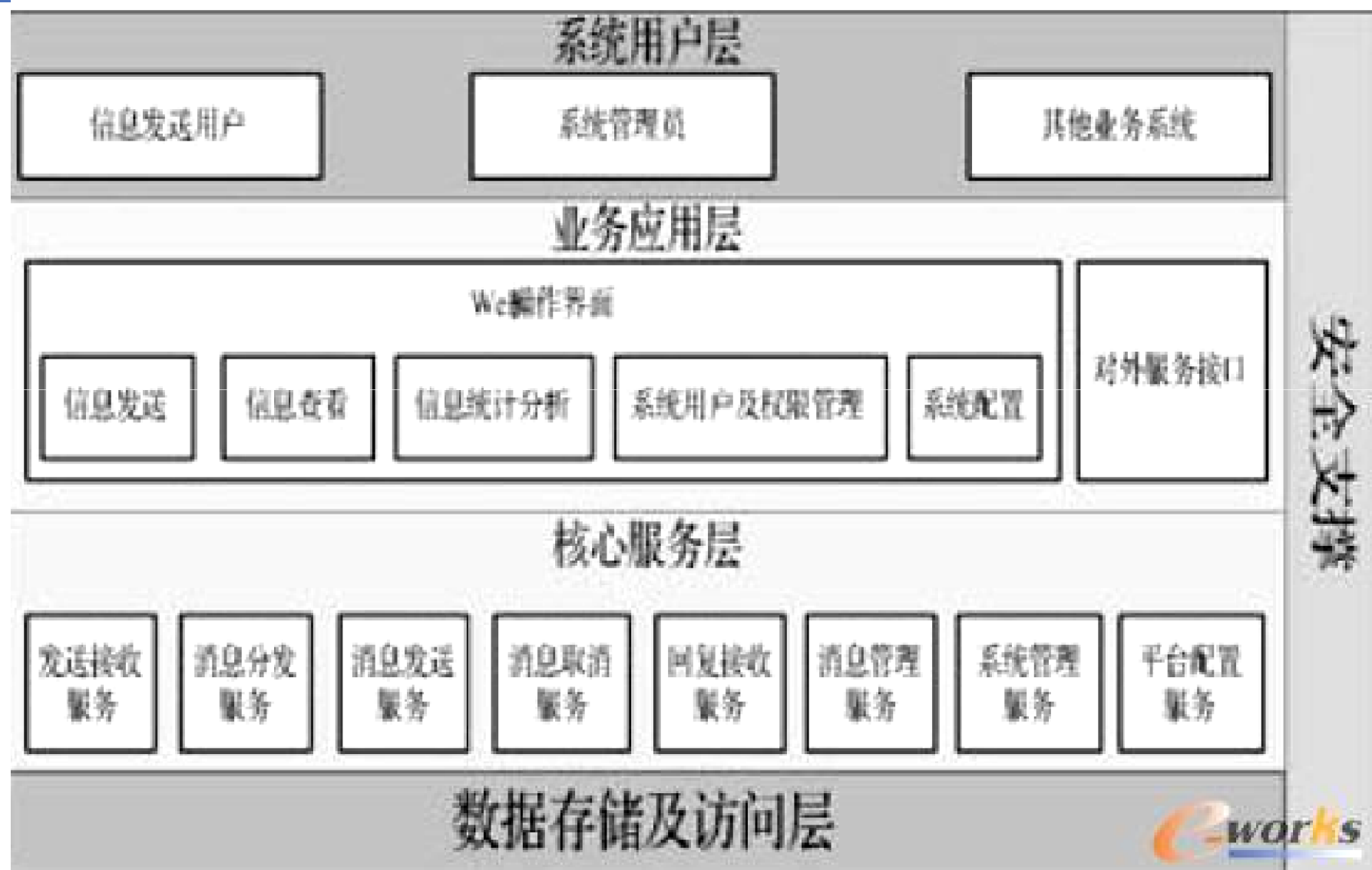
Architectural Design Steps

1. Design preliminary software architecture
2. Reuse available software resources
3. Refined architecture design
 - (1) Select the **public infrastructure** for the software architecture;
 - Database, Message oriented middleware, etc.
 - For some basic services, the infrastructure has already provided relevant functions
 - For those basic services whose infrastructure does not provide relevant functions and needs secondary development, the software architect needs to make further development

Architectural Design Steps

3. 细化架构设计

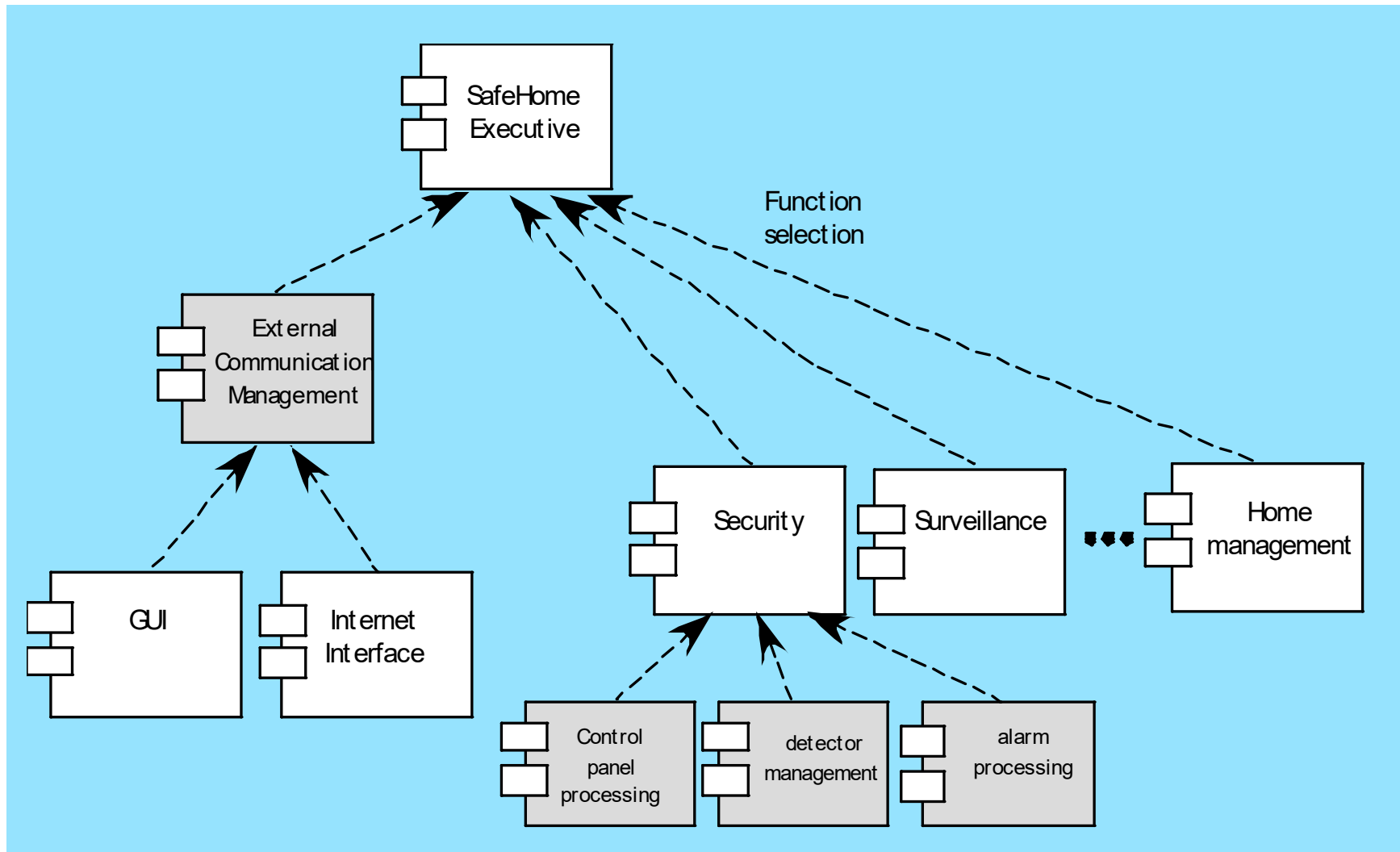
- 选择软件架构的公共基础设施：
 - 数据库、面向消息的中间件等。
 - 对于一些基本服务，基础设施已经提供了相关功能。
 - 对于那些基础设施不提供相关功能且需要二次开发的基本服务，软件架构师需要进一步开发。
- 对于所有软件需求，确定软件架构中的设计元素，定义它们的职责和接口。
 - 子系统、组件、类



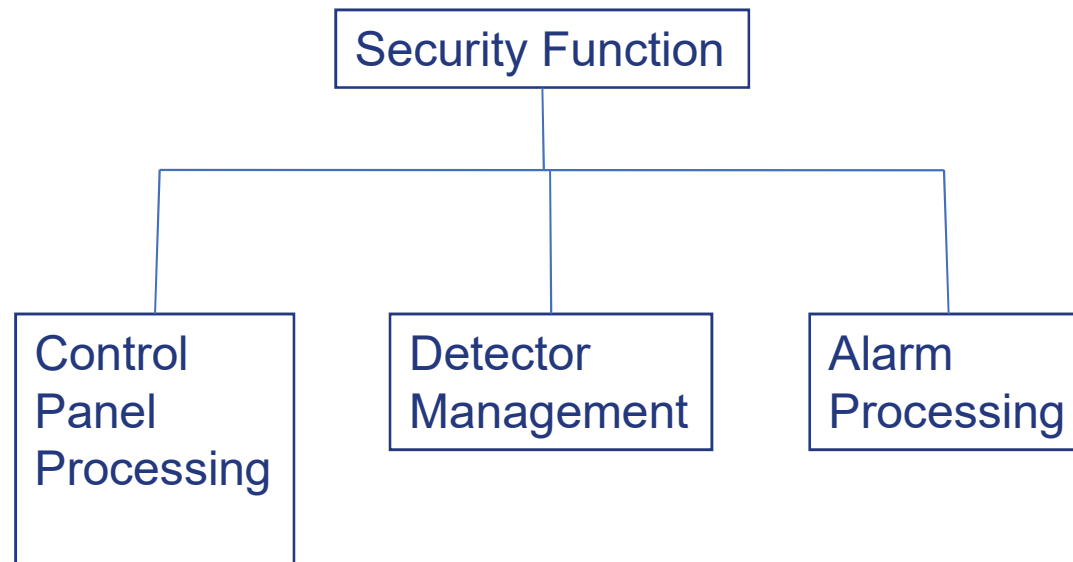
Architectural Design Steps

- ❖ 软件架构师可采用以下的策略和方法确定软件体系结构中的设计元素
- ❖ 1) 针对需求分析阶段所产生的用例集合，根据用例在业务方面的相关性或相似性对其进行归类，将同一类用例设计为一个子系统或构件，并根据用例的功能、参与用实现的分析类的职责两项因素来确定子系统的职责。
- ❖ 2) 针对需求分析阶段所产生的分析类模型，将具有相似或相关职责的控制类归为一类子系统或构件;也可将所有控制类的职责进行归并，按照业务上的相关性和相似性重新进行分组，每组的职责归并为一个子系统或构件。
- ❖ 3) 针对需求分析阶段所产生的实体类模型，将具有相类似或相关职责的实体类归为一类子系统或构件;也可将所有实体类的职责进行归并，按照业务上的相关性和相似性重新进行分组，每组的职责归并为一个子系统或构件。

Component Structure



Component Structure



Architectural Design Steps

1. Design preliminary software architecture
2. Reuse available software resources
3. Refined architecture design
4. Design deployment model
 - Indicates how software functionality and subsystems will be allocated within the physical computing environment
 - Modeled using UML deployment diagrams

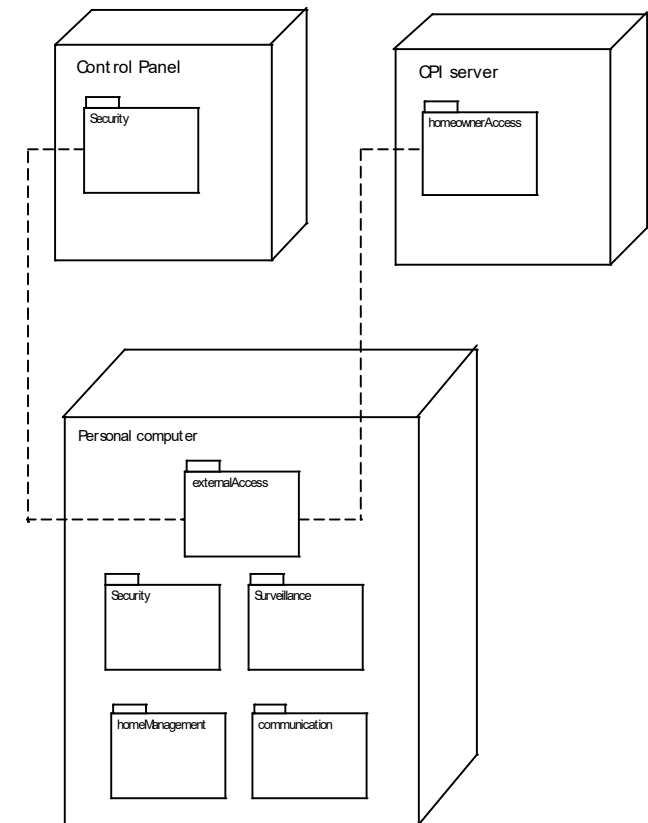


Figure 9.8 UML deployment diagram for SafeHome

4. 设计部署模型

- 表明如何在物理计算环境中分配软件功能和子系统。
- 使用UML部署图进行建模。

Architectural Design Steps

- 1. Design preliminary software architecture**
- 2. Reuse available software resources**
- 3. Refined architecture design**
- 4. Design deployment model**
- 5. Architecture documentation**

1. 设计初步的软件架构
2. 重用可用的软件资源
3. 细化架构设计
4. 设计部署模型
5. 架构文档编写

Documentation

1.2 项目风险	2
1.3 预期读者和阅读建议	2
1.4 参考资料	2
2. 设计概述	3
2.1 限制和约束	3
2.2 设计原则和设计要求	3
3. 系统逻辑设计	4
3.1 系统组织设计	4
3.2 系统结构设计	4
3.2.1 系统特性表	5
3.2.2 系统特性结构图	6
3.3 系统接口设计	6
3.3.1 系统接口表	6
3.3.2 系统接口传输协议说明	7
3.4 系统完整性设计	7
4. 系统出错处理设计	8
4.1 系统出错处理表	8
4.2 维护处理过程表	9
5. 技术设计	10
5.1 系统开发技术说明表	10
5.2 开发技术应用说明	11
6. 数据库设计	11
7. 词汇表	11

ADL

- ❖ *Architectural description language (ADL)* provides a semantics and syntax for describing a software architecture
- ❖ Provide the designer with the ability to:
 - decompose architectural components
 - compose individual components into larger architectural blocks and
 - represent interfaces (connection mechanisms) between components.

- 架构描述语言（ADL）提供了描述软件架构的语义和语法。
- 提供设计师能力：
 - 分解架构组件
 - 将各个组件组合成更大的架构块
 - 表示组件之间的接口（连接机制）

Architecture Reviews

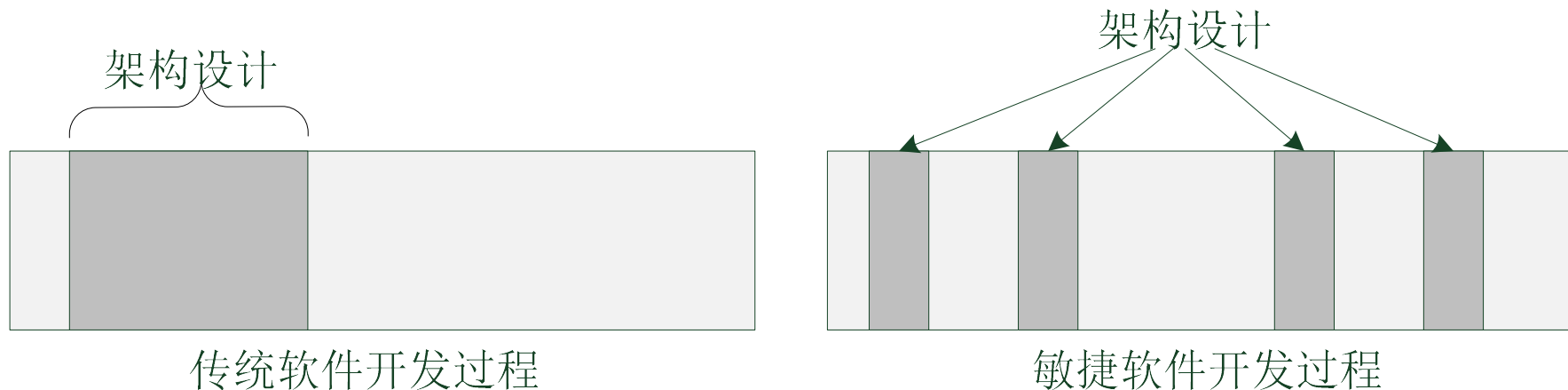
- ❖ **Assess (评估)** the ability of the software architecture to meet the systems quality requirements and identify potential risks.
- ❖ Have the potential to **reduce project costs** by detecting design problems early.
- ❖ Often make use of experience-based reviews, prototype evaluation, scenario reviews, and checklists.

- 评估（评估）软件架构满足系统质量要求的能力，并识别潜在风险。
- 通过及早发现设计问题，有可能降低项目成本。
- 通常利用基于经验的评审、原型评估、场景评审和检查表。

Agility and Architecture

❖ **To avoid rework, user stories are used to create and evolve an architectural model (walking skeleton) before coding.**

- 为避免返工，在编码前使用用户故事来创建和演变架构模型（行走骨架）。



7. 架构设计

- 软件的上下文

- 结构化设计

- 架构上下文图 (ACD)

- 上级系统

- 使用者

- 下级系统

- 架构原型

- 构件精化

8. 架构的考虑因素

- 经济——最好的软件是整洁的，依靠抽象来减少不必要的细节
- 可见性——架构决策及其原因对于稍后检查模型的软件工程师来说应该是显而易见的
- 间距——在设计中分离关注点而不引入隐藏的依赖关系
- 对称——架构对称意味着系统在其属性上是一致的和平衡的
- 涌现——涌现的、自组织的行为和控制

9. 架构设计文档

1. 确定每个决策需要哪些信息项
2. 定义每个决策和适当要求之间的联系
3. 当需要评估替代决策时，提供改变状态的机制
4. 定义决策之间的先决关系以支持可追溯性经常使用基于经验的评审、原型评估、情景评审和清单
5. 将重大决策与决策产生的架构视图联系起来
6. 记录并传达所有决策

10. 架构描述语言 (ADL) ——分解架构组件、将单个组件组合成更大的架构块、表示接口（组件之间的连接机制）

11. 架构评估

1. 评估软件架构满足系统质量要求和识别潜在风险的能力
2. 通过及早发现设计问题，有可能降低项目成本
3. 经常使用基于经验的评审、原型评估、情景评审和清单

12. 敏捷开发与架构

- 为了避免返工，用户故事用于在编码之前创建和发展架构模型
- 软件架构师将用户故事放在不断发展的故事板
- 运行良好的敏捷项目包括在每个冲刺期间交付工作产品
- 审查从冲刺中出现的代码可能是一种有用的架构审查形式



Southeast University



Thank You !

lliao@seu.edu.cn